# TP Client/serveur – GL2 : Utilisation De l'interface sockets pour des communications TCP/UDP de type client/serveur

## I.Introduction :

**Cette partie nous a expliqué lors de la communication entre serveur et client, comment on peut observer les échanges de paquets les messages http, les segments tcp et les datagrammes IP grâce à tcpdump ou wireshark .On va voir ça après dans le point 2.2**

**Pour notre travail,on a fait un lien git dans lequel vous pouvez consulter notre demarche.**

## II. Développement d'un client HTTP :

### 2.1-Vérification de la présence du serveur http :

**Dans cette partie, on va essayer de contacter le serveur http sur le port 80 en lui envoyant une requête dont le serveur va exécuter.**

**--Rq : On a essayé avec l'adresse IP dans le IP mais ça n'a pas marché et donc a essayé avec une autre adresse IP: par exemple celle de google.com**

**D'abord, on a fait sudo apt install telnet pour télécharger telnet car telnet nous permet de se connecter à un ordinateur ou un serveur à distance**

**--Avec HEAD :**

**Head permet juste d'afficher ce qui se trouve dans header et voilà un exemple pour google.com .**

**On a taper dans le terminal telnet google.com 80 puis quand il écrit connected to google.com, on va maintenant saisir notre requête http :**

**HEAD / http/1.1**
**Host :google.com**

**C'est-à-dire va à cette page de google.com et affiche moi le header**

**Voilà une capture d'écran pour notre exécution :**

```
eya@eya-VirtualBox: ~/Des...   eya@eya-VirtualBox: ~/Des...   eya@eya-VirtualBox: ~/Des...
eya@eya-VirtualBox:~/Desktop/TP_Sockets_GL2$ telnet google.com 80
Trying 142.250.181.174...
Connected to google.com.
Escape character is '^]'.
HEAD / HTTP/1.1
host:google.com

HTTP/1.1 301 Moved Permanently
Location: http://www.google.com/
Content-Type: text/html; charset=UTF-8
Content-Security-Policy-Report-Only: object-src 'none';base-uri 'self';script-sr
c 'nonce-3wd3K9HRTaufSIRH081TMQ' 'strict-dynamic' 'report-sample' 'unsafe-eval'
'unsafe-inline' https: http:;report-uri https://csp.withgoogle.com/csp/gws/other
-hp
Date: Thu, 09 Apr 2026 14:45:19 GMT
Expires: Sat, 09 May 2026 14:45:19 GMT
Cache-Control: public, max-age=2592000
Server: gws
Content-Length: 219
X-XSS-Protection: 0
X-Frame-Options: SAMEORIGIN


S
```

**--avec GET :**

**Get permet de récupérer le html et voilà un exemple pour google.com**

**On a fait encore telnet google.com 80 puis on a saisi notre requête http :**

**GET / http/1.1**

**Host :google.com**

**C'est-à-dire va à cette page de google.com et affiche moi le header**

**Voilà une capture d'écran pour notre exécution :**

```
SConnection closed by foreign host.
eya@eya-VirtualBox:~/Desktop/TP_Sockets_GL2$ telnet google.com 80
Trying 142.250.181.174...
Connected to google.com.
Escape character is '^]'.
GET / HTTP/1.1
host:google.com

HTTP/1.1 301 Moved Permanently
Location: http://www.google.com/
Content-Type: text/html; charset=UTF-8
Content-Security-Policy-Report-Only: object-src 'none';base-uri 'self';script-src 'nonce-iWrwhLBTuaNDVqkr3wN1qA' 'strict
-dynamic' 'report-sample' 'unsafe-eval' 'unsafe-inline' https: http:;report-uri https://csp.withgoogle.com/csp/gws/other
-hp
Date: Thu, 09 Apr 2026 15:03:05 GMT
Expires: Sat, 09 May 2026 15:03:05 GMT
Cache-Control: public, max-age=2592000
Server: gws
Content-Length: 219
X-XSS-Protection: 0
X-Frame-Options: SAMEORIGIN

<HTML><HEAD><meta http-equiv="content-type" content="text/html;charset=utf-8">
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
The document has moved
<A HREF="http://www.google.com/">here</A>.
</BODY></HTML>
```

**--Rq :**

**Quand on a fait dans telnet google.com et puis host :google.com, avec soit head ou get, on a remarqué que dans la requête réponse, il fait 301 moved permanently puis location [www.google.com](http://www.google.com).**

**En effet, google fait comme ça pour éviter plusieurs versions et pour rediriger tous les visiteurs vers le bon endroit. Voilà un exemple avec [www.google.com](http://www.google.com)**

```
eya@eya-VirtualBox:~$ telnet www.google.com 80
Trying 142.251.153.119...
Connected to www.google.com.
Escape character is '^]'.
HEAD / HTTP/1.1
host:www.google.com

HTTP/1.1 200 OK
Content-Type: text/html; charset=ISO-8859-1
Content-Security-Policy-Report-Only: object-src 'none';base-uri 'self';script-src 'nonce-IboFnhGnsVxsZsDZnp
-dynamic' 'report-sample' 'unsafe-eval' 'unsafe-inline' https: http:;report-uri https://csp.withgoogle.com/
-hp
P3P: CP="This is not a P3P policy! See g.co/p3phelp for more info."
Date: Thu, 09 Apr 2026 21:00:51 GMT
Server: gws
X-XSS-Protection: 0
X-Frame-Options: SAMEORIGIN
Expires: Thu, 09 Apr 2026 21:00:51 GMT
Cache-Control: private
Set-Cookie: AEC=AaJma5thx5vjPCuQRJxLpnPnC9FBCuNukNCF1z88cgL4Sytob7J4Ws8jFVw; expires=Tue, 06-Oct-2026 21:06
=/; domain=.google.com; Secure; HttpOnly; SameSite=lax
Set-Cookie: NID=530=iAou_6fM6Ym94Z9uQ8ok3QHBRo86Stn7XzBpsGMedxfKtL4BFVYBfkHdIQc4Tg-JD458YMc7nJbn0KkC5Qp7FAS
bE6gqahnPX70UBhzsEyVLUZW4SV4YLoLiOUJiMHOOdE6TFMLJNdMpXJNXU97NqWd4C2zoUrJZXq06g4YkNYETVACvKCCLcmvR0vmwZM4q0r
hQ; expires=Fri, 09-Oct-2026 21:00:51 GMT; path=/; domain=.google.com; HttpOnly
Transfer-Encoding: chunked

^C^C^C

Connection closed by foreign host.
eya@eya-VirtualBox:~$
eya@eya-VirtualBox:~$
```

## 2-2. Client HTTP en mode connecté :

**Dans cette partie, on va programmer notre code avec c en utilisant vs code.**

**On va Ecrire un programme client en mode connecté qui lit au clavier une requête HTTP écrite par l'utilisateur et affiche la réponse du serveur et voilà notre code :**

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <arpa/inet.h>

#include <netdb.h>


#define PORT 80

#define BUFFER_SIZE 4096

#define REQUEST_SIZE 8192


int main() {

   int sock;

   struct sockaddr_in server;

   char hostname[100];

   char request[REQUEST_SIZE];

   char buffer[BUFFER_SIZE];

   int bytes;


   // 1. Demander le serveur

   printf("Entrez le serveur (ex: google.com) : ");

   scanf("%s", hostname);


   // 2. Résolution DNS

   struct hostent *he = gethostbyname(hostname);

   if (he == NULL) {

      printf("Erreur: serveur introuvable\n");
```

```
    return 1;
}

// 3. Création socket
sock = socket(AF_INET, SOCK_STREAM, 0);
if (sock < 0) {
    perror("socket");
    return 1;
}

// 4. Configuration serveur
server.sin_family = AF_INET;
server.sin_port = htons(PORT);
memcpy(&server.sin_addr, he->h_addr_list[0], he->h_length);

// 5. Connexion
if (connect(sock, (struct sockaddr*)&server, sizeof(server)) < 0) {
    perror("connect");
    return 1;
}

printf("\nConnecté au serveur %s\n", hostname);

// vider le buffer après scanf
getchar();

// 6. Lecture requête HTTP
printf("\nTapez votre requête HTTP :\n");
printf("(Terminez par une ligne vide)\n\n");

request[0] = '\0';
```

```
    while (1) {
        char line[1024];

        if (fgets(line, sizeof(line), stdin) == NULL)
            break;

        if (strcmp(line, "\n") == 0) {
            strncat(request, "\r\n", REQUEST_SIZE - strlen(request) - 1);
            break;
        }

        strncat(request, line, REQUEST_SIZE - strlen(request) - 1);
    }

    // 7. Envoi
    send(sock, request, strlen(request), 0);

    printf("\n----- Requête envoyée -----\n%s\n", request);

    // 8. Réception
    printf("\n----- Réponse serveur -----\n");
    while ((bytes = recv(sock, buffer, BUFFER_SIZE - 1, 0)) > 0) {
        buffer[bytes] = '\0';
        printf("%s", buffer);
    }
// 9. Fermeture
    close(sock);
printf("\n\n----- Fin -----\n");
 return 0;}
```

**on a cree un fichier http_client.c et on a fait ces commandes pour compiler et executer notre fichier :**

**on a fait la requete GET / HTTP/1.1**

**host :google.com**

```
eya@eya-VirtualBox:~/Desktop/TP_Sockets_GL2/TP_Sockets_GL2$ gcc http_client.c -ohttp_client
eya@eya-VirtualBox:~/Desktop/TP_Sockets_GL2/TP_Sockets_GL2$ ./http_client
Entrez le serveur (ex: google.com) : google.com

Connecté au serveur google.com

Tapez votre requête HTTP :
(Terminez par une ligne vide)

GET / HTTP/1.1
host:google.com


----- Requête envoyée -----
GET / HTTP/1.1
host:google.com



----- Réponse serveur -----
HTTP/1.1 301 Moved Permanently
Location: http://www.google.com/
Content-Type: text/html; charset=UTF-8
Content-Security-Policy-Report-Only: object-src 'none';base-uri 'self';script-src 'nonce-wvsOBIQFeiCwh0s-k8
-dynamic' 'report-sample' 'unsafe-eval' 'unsafe-inline' https: http:;report-uri https://csp.withgoogle.com/
-hp
Date: Thu, 09 Apr 2026 16:49:16 GMT
Expires: Sat, 09 May 2026 16:49:16 GMT
Cache-Control: public, max-age=2592000
Server: gws
```

```
----- Requête envoyée -----
GET / HTTP/1.1
host:google.com



----- Réponse serveur -----
HTTP/1.1 301 Moved Permanently
Location: http://www.google.com/
Content-Type: text/html; charset=UTF-8
Content-Security-Policy-Report-Only: object-src 'none';base-uri 'self';script-src 'nonce-wvsOBIQFeiCwh0s-k8
-dynamic' 'report-sample' 'unsafe-eval' 'unsafe-inline' https: http:;report-uri https://csp.withgoogle.com/
-hp
Date: Thu, 09 Apr 2026 16:49:16 GMT
Expires: Sat, 09 May 2026 16:49:16 GMT
Cache-Control: public, max-age=2592000
Server: gws
Content-Length: 219
X-XSS-Protection: 0
X-Frame-Options: SAMEORIGIN

<HTML><HEAD><meta http-equiv="content-type" content="text/html;charset=utf-8">
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
The document has moved
<A HREF="http://www.google.com/">here</A>.
</BODY></HTML>
```

***Pour pouvoir capturer les paquets les messsages http, les segments TCP et les datagrammes IP, on va utiliser tcpdump ou wireshark**

**--tcpdump :On a d'abord installer tcpdump et on a tape dans le terminal :**

**sudo apt install tcpdump**

**Sudo tcpdump –i any port 80 puis on ouvre un autre terminal et on tape ./http_client et on voit ca :**

```
eya@eya-VirtualBox: ~/Desktop/TP_Sockets_G...   eya@eya-VirtualBox: ~/Desktop/TP_Sockets_G...
17:26:51.861608 enp0s3 Out IP eya-VirtualBox.34606 > arn09s21-in-f14.1e100.net.h
ttp: Flags [S], seq 416512559, win 64240, options [mss 1460,sackOK,TS val 366017
6828 ecr 0,nop,wscale 9], length 0
17:26:51.947926 enp0s3 In  IP arn09s21-in-f14.1e100.net.http > eya-VirtualBox.34
606: Flags [S.], seq 1248384001, ack 416512560, win 65535, options [mss 1460], l
ength 0
17:26:51.947958 enp0s3 Out IP eya-VirtualBox.34606 > arn09s21-in-f14.1e100.net.h
ttp: Flags [.], ack 1, win 64240, length 0
17:26:51.949599 enp0s3 Out IP eya-VirtualBox.34606 > arn09s21-in-f14.1e100.net.h
ttp: Flags [P.], seq 1:56, ack 1, win 64240, length 55: HTTP: GET / HTTP/1.1
17:26:51.949927 enp0s3 In  IP arn09s21-in-f14.1e100.net.http > eya-VirtualBox.34
606: Flags [.], ack 56, win 65535, length 0
17:26:52.063961 enp0s3 In  IP arn09s21-in-f14.1e100.net.http > eya-VirtualBox.34
606: Flags [P.], seq 1:793, ack 56, win 65535, length 792: HTTP: HTTP/1.1 301 Mo
ved Permanently
17:26:52.063983 enp0s3 Out IP eya-VirtualBox.34606 > arn09s21-in-f14.1e100.net.h
ttp: Flags [.], ack 793, win 63448, length 0
17:26:52.064710 enp0s3 In  IP arn09s21-in-f14.1e100.net.http > eya-VirtualBox.34
606: Flags [F.], seq 793, ack 56, win 65535, length 0
17:26:52.065593 enp0s3 Out IP eya-VirtualBox.34606 > arn09s21-in-f14.1e100.net.h
ttp: Flags [F.], seq 56, ack 794, win 63447, length 0
17:26:52.065744 enp0s3 In  IP arn09s21-in-f14.1e100.net.http > eya-VirtualBox.34
606: Flags [.], ack 57, win 65535, length 0
```

**Et dans le nouveau terminal :**

```
eya@eya-VirtualBox:~/Desktop/TP_Sockets_GL2/TP_Sockets_GL2$ ./http_client
Connecté au serveur
Requête envoyée :
GET / HTTP/1.1
Host: google.com
Connection: close


HTTP/1.1 301 Moved Permanently
Location: http://www.google.com/
Content-Type: text/html; charset=UTF-8
Content-Security-Policy-Report-Only: object-src 'none';base-uri 'self';script-src 'nonce-1LbGNM33yQ0MXuZ6Br
-dynamic' 'report-sample' 'unsafe-eval' 'unsafe-inline' https: http:;report-uri https://csp.withgoogle.com/
-hp
Date: Thu, 09 Apr 2026 16:26:51 GMT
Expires: Sat, 09 May 2026 16:26:51 GMT
Cache-Control: public, max-age=2592000
Server: gws
Content-Length: 219
X-XSS-Protection: 0
X-Frame-Options: SAMEORIGIN
Connection: close

<HTML><HEAD><meta http-equiv="content-type" content="text/html;charset=utf-8">
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
The document has moved
<A HREF="http://www.google.com/">here</A>.
</BODY></HTML>
eya@eya-VirtualBox:~/Desktop/TP_Sockets_GL2/TP_Sockets_GL2$
```

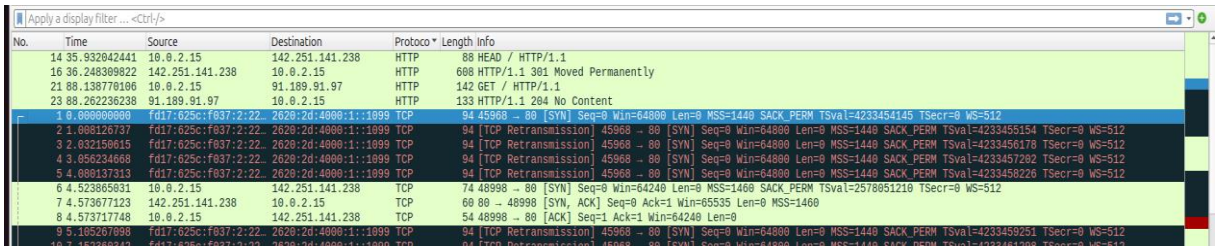
**--wireshark :on doit d'abord installer wireshark et on tape dans terminal :**

**Sudo apt install wireshark puis on lance wireshark**

**Ici les deux premiéres lignes montrent l'echange de HTTP observé dans wireshark**

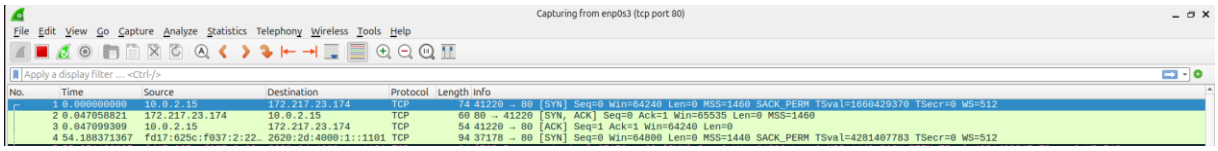
**Première ligne : envoi de la requête GET HTTP 1.1**

**Deuxième ligne : réponse du serveur**



| No. | Time | Source | Destination | Protoco | Length | Info |
|---|---|---|---|---|---|---|
| 14 | 35.932042441 | 10.0.2.15 | 142.251.141.238 | HTTP | 88 | HEAD / HTTP/1.1 |
| 16 | 36.248309822 | 142.251.141.238 | 10.0.2.15 | HTTP | 608 | HTTP/1.1 301 Moved Permanently |
| 21 | 88.138770106 | 10.0.2.15 | 91.189.91.97 | HTTP | 142 | GET / HTTP/1.1 |
| 23 | 88.262236238 | 91.189.91.97 | 10.0.2.15 | HTTP | 133 | HTTP/1.1 204 No Content |
| 1 | 0.000000000 | fd17:625c:f037:2:22… | 2620:2d:4000:1::1099 | TCP | 94 | 45968 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4233454145 TSecr=0 WS=512 |
| 2 | 1.008126737 | fd17:625c:f037:2:22… | 2620:2d:4000:1::1099 | TCP | 94 | [TCP Retransmission] 45968 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4233455154 TSecr=0 WS=512 |
| 3 | 2.032150615 | fd17:625c:f037:2:22… | 2620:2d:4000:1::1099 | TCP | 94 | [TCP Retransmission] 45968 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4233456178 TSecr=0 WS=512 |
| 4 | 3.056234668 | fd17:625c:f037:2:22… | 2620:2d:4000:1::1099 | TCP | 94 | [TCP Retransmission] 45968 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4233457202 TSecr=0 WS=512 |
| 5 | 4.088137313 | fd17:625c:f037:2:22… | 2620:2d:4000:1::1099 | TCP | 94 | [TCP Retransmission] 45968 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4233458226 TSecr=0 WS=512 |
| 6 | 4.523865631 | 10.0.2.15 | 142.251.141.238 | TCP | 74 | 48998 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2578051210 TSecr=0 WS=512 |
| 7 | 4.573677123 | 142.251.141.238 | 10.0.2.15 | TCP | 60 | 80 → 48998 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 |
| 8 | 4.573717748 | 10.0.2.15 | 142.251.141.238 | TCP | 54 | 48998 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0 |
| 9 | 5.105267098 | fd17:625c:f037:2:22… | 2620:2d:4000:1::1099 | TCP | 94 | [TCP Retransmission] 45968 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4233459251 TSecr=0 WS=512 |

**Ici dans cette photo en dessous on voit l'établissement de la connexion TCP entre le client et le serveur HTTP :**



| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 1 | 0.000000000 | 10.0.2.15 | 172.217.23.174 | TCP | 74 | 41220 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1660429370 TSecr=0 WS=512 |
| 2 | 0.047058821 | 172.217.23.174 | 10.0.2.15 | TCP | 60 | 80 → 41220 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 |
| 3 | 0.047099309 | 10.0.2.15 | 172.217.23.174 | TCP | 54 | 41220 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0 |
| 4 | 54.188371367 | fd17:625c:f037:2:22… | 2620:2d:4000:1::1101 | TCP | 94 | 37178 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4281407783 TSecr=0 WS=512 |

**Maintenant, on va répondre à ces questions :**

## 1- Comment s'établit la connexion au niveau HTTP et au niveau TCP ?

**.au niveau tcp : la connexion s'établit par 3 way handshake**

**Client → SYN → Serveur**
**Serveur → SYN-ACK → Client**
**Client → ACK → Serveur**

**le client synchronise avec serveur et demande ouverture de connexion .Le serveur accepte et accuse réception puis le client confirme la connexion est établie.**

**.au niveau http : http fonctionne selon requête +réponse. Le client envoie une requête et le serveur génère la réponse.**

## 2- Qui décide de fermer la connexion ?

**.Pour la fermeture, la connexion est fermée généralement par le serveur puis le client ferme aussi grâce a close(sock)**

**[sock :c'est un numéro qui représente une connexion réseau ,c'est une référence.] et [socket :c'est un point de communication et permet a 2 progs de communiquer via le réseau.]**

## 3-Quand ?

**. Après l'envoi complet de la réponse http.**

## 4-Quels sont les ports utilisés par le client et le serveur ?

.le client va choisir automatiquement un port libre et pour le serveur on utilise port 80.

## 5-un chronogramme des échanges observés :

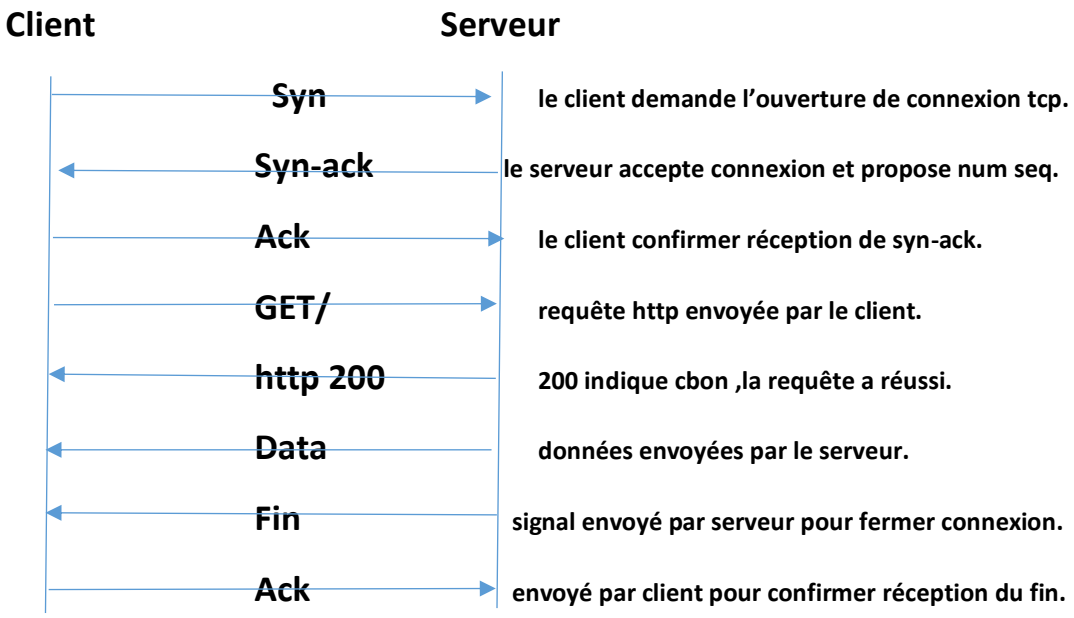
--explication des termes du diagramme :

Syn : Synchronize

Syn+Ack : Synchronize + Acknowledge

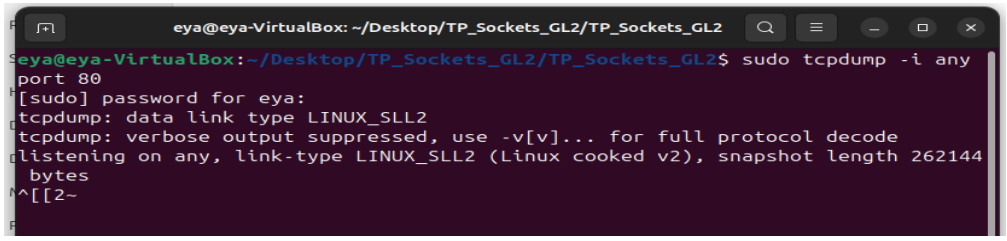
Ack :acknowladge

--Diagramme :




## 6-Refaites la même capture en remplaçant votre client HTTP par celui de la question 2.1. Que constatez-vous ? Expliquez !

.Dans premier terminal :




Puis dans un autre terminal on fait comme dans 2.1:

```
23:48:31.739279 enp0s3 In  IP6 connectivity-check-ubuntu-com-2.ps5.canonical.com
.http > eya-VirtualBox.39702: Flags [R.], seq 0, ack 191567137, win 65535, lengt
h 0
23:48:31.739398 enp0s3 Out IP6 eya-VirtualBox.35364 > connectivity-check-ubuntu-
com-1.ps5.canonical.com.http: Flags [S], seq 3779223035, win 64800, options [mss
 1440,sackOK,TS val 1282731866 ecr 0,nop,wscale 9], length 0
23:48:31.750391 enp0s3 In  IP6 connectivity-check-ubuntu-com-1.ps5.canonical.com
.http > eya-VirtualBox.35364: Flags [R.], seq 0, ack 3779223036, win 65535, leng
th 0
23:48:31.750588 enp0s3 Out IP6 eya-VirtualBox.39108 > ubuntu-content-cache-2.ps6
.canonical.com.http: Flags [S], seq 385803624, win 64800, options [mss 1440,sack
OK,TS val 2693641067 ecr 0,nop,wscale 9], length 0
23:48:31.751062 enp0s3 In  IP6 ubuntu-content-cache-2.ps6.canonical.com.http > e
ya-VirtualBox.39108: Flags [R.], seq 0, ack 385803625, win 65535, length 0
23:48:32.739777 enp0s3 Out IP eya-VirtualBox.50028 > ncmrsa-aq-in-f14.1e100.net.
http: Flags [P.], seq 33:35, ack 1, win 64240, length 2: HTTP
23:48:32.740301 enp0s3 In  IP ncmrsa-aq-in-f14.1e100.net.http > eya-VirtualBox.5
0028: Flags [.], ack 35, win 65535, length 0
23:48:32.874936 enp0s3 In  IP ncmrsa-aq-in-f14.1e100.net.http > eya-VirtualBox.5
0028: Flags [P.], seq 1:587, ack 35, win 65535, length 586: HTTP: HTTP/1.1 301 M
oved Permanently
```

**Avec telnet , plusieurs petits paquets tcp sont envoyes car on tape manuellement ca dans le terminal pas comme en c.**

# III. Transfert de messages en mode connecté

**.but de la partie 3 :**

**Développer une application client/serveur en mode connecté TCP où le client envoie 60 fois l'heure courante au serveur.**

**.Observation attendue :**

**Le protocole TCP assure une transmission fiable et ordonnée. Ainsi, même si le serveur traite les messages plus lentement avec sleep(1), il ne devrait pas y avoir de perte de messages au niveau de l'application. Les segments observés avec Wireshark ne correspondent pas nécessairement un-à-un aux appels send() du programme.**

**Le serveur doit : attendre une connexion d'un client, accepte cette connexion, reçoit les messages envoyées , affiche les messages et puis compte combien de messages il a reçu.**

**Le client doit :créer socket tcp, se connecte au serveur, envoyer bonjour, envoyer 60 fois l'heure courante, envoyer au revoir affiche combien de messages il a envoyé.**

**--code du serveur :**

```
#include <stdio.h>

#include <stdlib.h>
```

```
#include <string.h>

#include <unistd.h>

#include <arpa/inet.h>

#include <sys/socket.h>


#define PORT 12345

#define BUFFER_SIZE 1024


int main() {

    int sock_serveur, sock_client;

    struct sockaddr_in addr_serveur, addr_client;

    socklen_t taille_client;

    char buffer[BUFFER_SIZE];

    int nb_recu;

    int compteur = 0;


    /* 1) Création de la socket TCP du serveur */

    sock_serveur = socket(AF_INET, SOCK_STREAM, 0);

    if (sock_serveur < 0) {

        perror("Erreur socket");

        return 1;

    }
```

```
/* 2) Préparation de l'adresse du serveur */
memset(&addr_serveur, 0, sizeof(addr_serveur));
addr_serveur.sin_family = AF_INET;
addr_serveur.sin_port = htons(PORT);
addr_serveur.sin_addr.s_addr = INADDR_ANY;

/* 3) Association de la socket à l'adresse et au port */
if (bind(sock_serveur, (struct sockaddr *)&addr_serveur, sizeof(addr_serveur)) < 0) {
    perror("Erreur bind");
    close(sock_serveur);
    return 1;
}

/* 4) Mise en écoute */
if (listen(sock_serveur, 5) < 0) {
    perror("Erreur listen");
    close(sock_serveur);
    return 1;
}

printf("Serveur en attente sur le port %d...\n", PORT);

/* 5) Acceptation d'un client */
taille_client = sizeof(addr_client);
```

```
sock_client = accept(sock_serveur, (struct sockaddr *)&addr_client, &taille_client);
if (sock_client < 0) {
    perror("Erreur accept");
    close(sock_serveur);
    return 1;
}

printf("Client connecte !\n");

/* 6) Réception des messages */
while ((nb_recu = recv(sock_client, buffer, BUFFER_SIZE - 1, 0)) > 0) {
    buffer[nb_recu] = '\0';
    compteur++;
    printf("Message %d recu : %s", compteur, buffer);
}

if (nb_recu < 0) {
    perror("Erreur recv");
}

printf("\nNombre total de messages recus : %d\n", compteur);

/* 7) Fermeture */
close(sock_client);
```

```
    close(sock_serveur);


    return 0;
}
```

## --code client :

```
include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <time.h>

#define SERVER_IP "127.0.0.1"
#define PORT 12345
#define BUFFER_SIZE 1024

int main() {
    int sock_client;
    struct sockaddr_in addr_serveur;
    char message[BUFFER_SIZE];
    int compteur = 0;
    time_t t;
    struct tm *tm_info;
```

```
/* 1) Création de la socket TCP */
sock_client = socket(AF_INET, SOCK_STREAM, 0);
if (sock_client < 0) {
    perror("Erreur socket");
    return 1;
}

/* 2) Préparation de l'adresse du serveur */
memset(&addr_serveur, 0, sizeof(addr_serveur));
addr_serveur.sin_family = AF_INET;
addr_serveur.sin_port = htons(PORT);

if (inet_pton(AF_INET, SERVER_IP, &addr_serveur.sin_addr) <= 0) {
    perror("Erreur adresse IP");
    close(sock_client);
    return 1;
}

/* 3) Connexion au serveur */
if (connect(sock_client, (struct sockaddr *)&addr_serveur, sizeof(addr_serveur)) < 0) {
    perror("Erreur connect");
    close(sock_client);
    return 1;
```

```
}

printf("Connecte au serveur %s:%d\n", SERVER_IP, PORT);

/* 4) Envoi de Bonjour */
strcpy(message, "Bonjour\n");
send(sock_client, message, strlen(message), 0);
compteur++;

/* 5) Envoi 60 fois de l'heure courante */
for (int i = 0; i < 60; i++) {
   t = time(NULL);
   tm_info = localtime(&t);

   strftime(message, BUFFER_SIZE, "Il est %H:%M:%S !\n", tm_info);

   send(sock_client, message, strlen(message), 0);
   compteur++;
}

/* 6) Envoi de Au revoir */
strcpy(message, "Au revoir\n");
send(sock_client, message, strlen(message), 0);
compteur++;
```

```
printf("Nombre total de messages envoyes : %d\n", compteur);

/* 7) Fermeture */

close(sock_client);

return 0;}
```

**Maintenant, on va répondre à ces questions :**

**1-Comptez sur le client et sur le serveur le nombre de messages échangés. Refaites le même comptage en ajoutant sur le serveur un délai d'environ une seconde entre chaque message (sleep). Que constatez-vous ? Y a-t-il des pertes de messages ?**

**Côté client, nous avons compté 62 messages envoyés : un message Bonjour, 60 messages contenant l'heure courante et un message Au revoir.**

**Côté serveur, le nombre affiché n'était pas égal à 62 il était égal à 11, car nous comptions le nombre d'appels à recv() et non pas le nombre réel de messages applicatifs. En effet, TCP est un protocole en flux : plusieurs appels send() du client peuvent être regroupés et reçus en une seule lecture côté serveur. C'est pourquoi on voit seulement 11 messages car ils étaient regroupés.**

**Après ajout d'un délai de 1 seconde avec sleep(1) dans le serveur, nous avons constaté que le nombre d'appels à recv() pouvait changer maintenant le nombre affiché était égal à 3, mais cela ne signifie pas qu'il y a des pertes. Les données sont toujours transmises de manière fiable en TCP.**
**Donc, il n'y a pas forcément de pertes de messages, mais un regroupement des données dans le flux TCP.**

**Exécution du serveur TCP : affichage des données reçues**

```
vboxuser@Ubuntu: ~/Deskt...   vboxuser@Ubuntu: ~/Deskt...   vboxuser@Ubuntu: ~/Deskt...
vboxuser@Ubuntu:~/Desktop/TP_Sockets_GL2$ gcc serveur_tcp.c -o serveur_tcp
vboxuser@Ubuntu:~/Desktop/TP_Sockets_GL2$ ./serveur_tcp
Serveur en attente sur le port 12345...
Client connecte !
Message 1 recu : Bonjour
Message 2 recu : Il est 09:03:54 !
Message 3 recu : Il est 09:03:54 !
Message 4 recu : Il est 09:03:54 !
Message 5 recu : Il est 09:03:54 !
Message 6 recu : Il est 09:03:54 !
Message 7 recu : Il est 09:03:54 !
Message 8 recu : Il est 09:03:54 !
Message 9 recu : Il est 09:03:54 !
Message 10 recu : Il est 09:03:54 !
Il est 09:03:54 !
Il est 09:03:54 !
Il est 09:03:54 !
Il est 09:03:54 !
Il est 09:03:54 !
Il est 09:03:54 !
Il est 09:03:54 !
Message 11 recu : Il est 09:03:54 !
Il est 09:03:54 !
Il est 09:03:54 !
```

```
Il est 09:03:54 !
Il est 09:03:54 !
Il est 09:03:54 !
Il est 09:03:54 !
Il est 09:03:54 !
Il est 09:03:54 !
Il est 09:03:54 !
Il est 09:03:54 !
Il est 09:03:54 !
Il est 09:03:54 !
Il est 09:03:54 !
Au revoir

Nombre total de messages recus : 11
vboxuser@Ubuntu:~/Desktop/TP_Sockets_GL2$
```

**Exécution du client TCP : 62 messages envoyés**

```
vboxuser@Ubuntu:~/Desktop/TP_Sockets_GL2$ gcc client_tcp.c -o client_tcp
vboxuser@Ubuntu:~/Desktop/TP_Sockets_GL2$ ./client_tcp
Connecte au serveur 127.0.0.1:12345
Nombre total de messages envoyes : 62
vboxuser@Ubuntu:~/Desktop/TP_Sockets_GL2$
```

**Même test après ajout de `sleep(1)` sur le serveur, Cote serveur:**

```
vboxuser@Ubuntu: ~/Deskt...   vboxuser@Ubuntu: ~/Deskt...   vboxuser@Ubuntu: ~/Deskt...
vboxuser@Ubuntu:~/Desktop/TP_Sockets_GL2$ gcc serveur_tcp_sleep.c -o serveur_tcp
_sleep
vboxuser@Ubuntu:~/Desktop/TP_Sockets_GL2$ ./serveur_tcp_sleep
Serveur en attente sur le port 12345...
Client connecte !
Message 1 recu :
Bonjour
Message 2 recu :
Il est 09:14:16 !
Il est 09:14:16 !
Il est 09:14:16 !
Il est 09:14:16 !
Il est 09:14:16 !
Il est 09:14:16 !
Il est 09:14:16 !
Il est 09:14:16 !
Il est 09:14:16 !
Il est 09:14:16 !
Il est 09:14:16 !
Il est 09:14:16 !
Il est 09:14:16 !
Il est 09:14:16 !
Il est 09:14:16 !
Il est 09:14:16 !
```

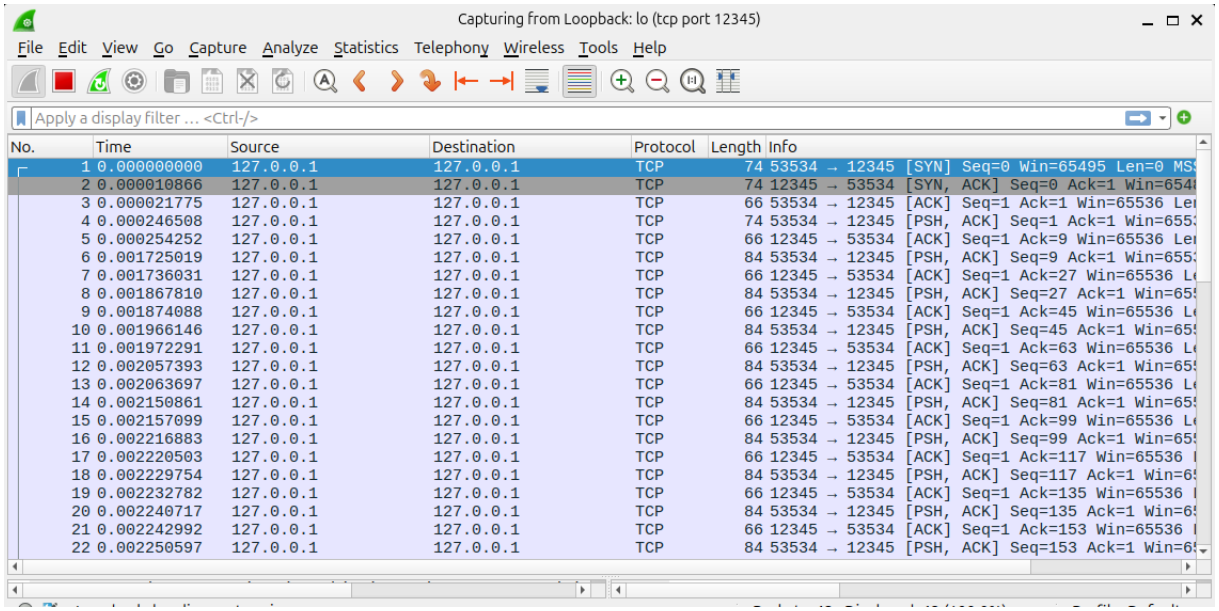
```
Il est 09:14:16 !
Il est 09:14:16 !
Il est 09:14:16 !
Il est 09:14:16 !
Il est 09:14:16Message 3 recu :
 !
Il est 09:14:16 !
Il est 09:14:16 !
Il est 09:14:16 !
Au revoir

Nombre total de messages recus : 3
vboxuser@Ubuntu:~/Desktop/TP_Sockets_GL2$
```

## 2-Regardez avec Wireshark les messages échangés. Les segments TCP envoyés correspondent-ils exactement aux écritures que votre programme fait sur la socket ?

**Non, les segments TCP ne correspondent pas exactement aux écritures faites par le programme sur la socket.**

**On n'a pas 62 segments TCP de données pour les 62 messages envoyés, car TCP ne travaille pas message par message mais comme un flux de données. Donc plusieurs `send()` du programme peuvent être regroupés dans un seul segment TCP, ou parfois découpés autrement. En plus, dans la capture, il y a aussi des paquets de contrôle comme SYN, ACK et `FIN` qui servent à établir et fermer la connexion, et qui ne correspondent pas à des messages de notre application.**

# Wireshark :




| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 1 | 0.000000000 | 127.0.0.1 | 127.0.0.1 | TCP | 74 | 53534 → 12345 [SYN] Seq=0 Win=65495 Len=0 MS |
| 2 | 0.000010866 | 127.0.0.1 | 127.0.0.1 | TCP | 74 | 12345 → 53534 [SYN, ACK] Seq=0 Ack=1 Win=654 |
| 3 | 0.000021775 | 127.0.0.1 | 127.0.0.1 | TCP | 66 | 53534 → 12345 [ACK] Seq=1 Ack=1 Win=65536 Le |
| 4 | 0.000246508 | 127.0.0.1 | 127.0.0.1 | TCP | 74 | 53534 → 12345 [PSH, ACK] Seq=1 Ack=1 Win=655 |
| 5 | 0.000254252 | 127.0.0.1 | 127.0.0.1 | TCP | 66 | 12345 → 53534 [ACK] Seq=1 Ack=9 Win=65536 Le |
| 6 | 0.001725019 | 127.0.0.1 | 127.0.0.1 | TCP | 84 | 53534 → 12345 [PSH, ACK] Seq=9 Ack=1 Win=655 |
| 7 | 0.001736031 | 127.0.0.1 | 127.0.0.1 | TCP | 66 | 12345 → 53534 [ACK] Seq=1 Ack=27 Win=65536 L |
| 8 | 0.001867810 | 127.0.0.1 | 127.0.0.1 | TCP | 84 | 53534 → 12345 [PSH, ACK] Seq=27 Ack=1 Win=65 |
| 9 | 0.001874088 | 127.0.0.1 | 127.0.0.1 | TCP | 66 | 12345 → 53534 [ACK] Seq=1 Ack=45 Win=65536 L |
| 10 | 0.001966146 | 127.0.0.1 | 127.0.0.1 | TCP | 84 | 53534 → 12345 [PSH, ACK] Seq=45 Ack=1 Win=65 |
| 11 | 0.001972291 | 127.0.0.1 | 127.0.0.1 | TCP | 66 | 12345 → 53534 [ACK] Seq=1 Ack=63 Win=65536 L |
| 12 | 0.002057393 | 127.0.0.1 | 127.0.0.1 | TCP | 84 | 53534 → 12345 [PSH, ACK] Seq=63 Ack=1 Win=65 |
| 13 | 0.002063697 | 127.0.0.1 | 127.0.0.1 | TCP | 66 | 12345 → 53534 [ACK] Seq=1 Ack=81 Win=65536 L |
| 14 | 0.002150861 | 127.0.0.1 | 127.0.0.1 | TCP | 84 | 53534 → 12345 [PSH, ACK] Seq=81 Ack=1 Win=65 |
| 15 | 0.002157099 | 127.0.0.1 | 127.0.0.1 | TCP | 66 | 12345 → 53534 [ACK] Seq=1 Ack=99 Win=65536 L |
| 16 | 0.002216883 | 127.0.0.1 | 127.0.0.1 | TCP | 84 | 53534 → 12345 [PSH, ACK] Seq=99 Ack=1 Win=65 |
| 17 | 0.002220503 | 127.0.0.1 | 127.0.0.1 | TCP | 66 | 12345 → 53534 [ACK] Seq=1 Ack=117 Win=65536 |
| 18 | 0.002229754 | 127.0.0.1 | 127.0.0.1 | TCP | 84 | 53534 → 12345 [PSH, ACK] Seq=117 Ack=1 Win=6 |
| 19 | 0.002232782 | 127.0.0.1 | 127.0.0.1 | TCP | 66 | 12345 → 53534 [ACK] Seq=1 Ack=135 Win=65536 |
| 20 | 0.002240717 | 127.0.0.1 | 127.0.0.1 | TCP | 84 | 53534 → 12345 [PSH, ACK] Seq=135 Ack=1 Win=6 |
| 21 | 0.002242992 | 127.0.0.1 | 127.0.0.1 | TCP | 66 | 12345 → 53534 [ACK] Seq=1 Ack=153 Win=65536 |
| 22 | 0.002250597 | 127.0.0.1 | 127.0.0.1 | TCP | 84 | 53534 → 12345 [PSH, ACK] Seq=153 Ack=1 Win=6 |

### 3-Que se passe-t-il si vous débranchez le câble réseau reliant le client et le serveur ? Même question si vous débranchez puis rebranchez rapidement le câble.

**Lorsque le câble réseau est débranché pendant la communication, la connexion TCP est perturbée. Les appels `send()` ou `recv()` peuvent se bloquer pendant un certain temps, puis retourner une erreur si la communication ne peut pas être rétablie à mais si le câble est rebranché rapidement, la communication peut parfois reprendre, sinon la connexion finit par être rompue.**

### 4-Pouvez-vous avec votre implémentation actuelle servir plusieurs clients ? Si oui, peuvent-ils être servis simultanément ? Lancez simultanément un nombre suffisant de clients pour remplir la file des connexions pendantes du serveur. Que se passe-t-il si vous lancez alors un client supplémentaire alors que la file est pleine ?

**Avec notre implémentation actuelle, le serveur ne peut pas servir plusieurs clients simultanément car on l'a programmé d'une manière séquentielle accept() après recv() donc il reste occupé avec le client dans la boucle recv() et il ne retourne pas à accept() pour prendre un autre client. Lorsqu'on lance plusieurs clients en même temps, certains parviennent à se connecter, tandis que d'autres reçoivent le message `Connection refused`. Cela montre que le serveur traite les clients de manière limitée.**

**Si cette file est pleine, un client supplémentaire peut être refusé ou attendre selon le comportement du système. Le serveur n'est donc pas encore concurrent, il ne peut pas servir plusieurs clients simultanément.**

**Lancement simultané de plusieurs clients : certaines connexions sont acceptées, d'autres refusées**

```
vboxuser@Ubuntu:~/Desktop/TP_Sockets_GL2$ ./client_tcp & ./client_tcp & ./client
_tcp & ./client_tcp & ./client_tcp &
[1] 10206
[2] 10207
[3] 10208
[4] 10209
[5] 10210
vboxuser@Ubuntu:~/Desktop/TP_Sockets_GL2$ Connecte au serveur 127.0.0.1:12345
Nombre total de messages envoyes : 62
Connecte au serveur 127.0.0.1:12345
Nombre total de messages envoyes : 62
Connecte au serveur 127.0.0.1:12345
Nombre total de messages envoyes : 62
Erreur connect: Connection refused
Connecte au serveur 127.0.0.1:12345
```